

ESP32forth et l'interface WiFi

Marc PETREMANN



Version 1.0 - 17/05/26

Table des matières

Préambule.....	3
ESP32 et son interface WiFi.....	4
Étapes d'une connexion complète.....	5
Initialisation du mode Station.....	5
Le vocabulaire WiFi.....	6
WIFI_MODE_AP (-- 2).....	6
WIFI_MODE_APSTA (-- 3).....	7
WIFI_MODE_NULL (-- 0).....	8
WIFI_MODE_STA.....	9
WiFi.begin (ssid-z password-z --).....	9
WiFi.config (ip dns gateway subnet --).....	9
WiFi.disconnect (--).....	9
WiFi.enableIPv6.....	10
WiFi.localIP (-- ip).....	10
WiFi.macAddress (addr --).....	10
Utilité.....	11
WiFi.mode (n --).....	12
WiFi.getTxPower (-- powerx4).....	13
WiFi.setTxPower (powerx4 --).....	13
WiFi.softAP.....	13
WiFi.softAPBroadcastIP.....	13
WiFi.softAPConfig.....	14
WiFi.softAPdisconnect.....	14
WiFi.softAPIP.....	14
WiFi.softAPNetworkID.....	14
WiFi.softAPgetStationNum.....	14
WiFi.status.....	14
WiFi.linkLocalIPv6.....	14
WiFi.globalIPv6.....	14
Index lexical.....	15

Préambule

Ce manuel est destiné à la prise en main de l'interface WiFi pour ESP32forth

ESP32 et son interface WiFi

L'interface WiFi de l'ESP32 est un sous-système puissant qui permet au microcontrôleur de communiquer sans fil via la pile TCP/IP. Sous **ESP32forth**, cette interface est pilotée par des mots qui font le pont avec les bibliothèques C++ natives. Elle fonctionne principalement selon trois modes : **Station (STA)** pour se connecter à un routeur, **Access Point (AP)** pour créer son propre réseau, ou le mode hybride **AP+STA**.

Le processus suit une logique séquentielle : on définit d'abord le mode avec **WiFi.mode**, puis on initie la connexion ou le service avec **WiFi.begin** ou **WiFi.softAP**. Comme la connexion est **asynchrone**, le programme doit surveiller l'état via **WiFi.status** avant d'échanger des données. Une fois connecté, l'ESP32 obtient une adresse IP (récupérable avec **WiFi.localIP**) et peut ouvrir des **sockets** pour agir comme un client ou un serveur Web, permettant ainsi de piloter du matériel à distance via un simple navigateur ou des requêtes HTTP.

Le WiFi 4 (Standard 2,4 GHz) est le standard pour la grande majorité des projets.

- **ESP32 Original (Série WROOM/WROVER)** : Le plus courant. WiFi + Bluetooth Classic + BLE.
- **ESP32-S2** : WiFi uniquement (pas de Bluetooth). Très économe en énergie.
- **ESP32-S3** : WiFi + Bluetooth LE. Très puissant, idéal pour l'IA et les écrans.
- **ESP32-C3** : WiFi + Bluetooth LE. Modèle économique à un seul cœur (architecture RISC-V).

Les modèles "Next Gen" (WiFi 6), sont plus récents (2024-2026) et offrent une meilleure gestion des réseaux encombrés :

- **ESP32-C6** : WiFi 6 + Bluetooth 5 + Zigbee/Thread (pour la domotique "Matter").
- **ESP32-C5** : C'est le premier modèle **Dual-Band** (2,4 GHz et **5 GHz**).

L'exception (Sans WiFi), une série récente fait exception :

- **ESP32-P4** : Cette carte est ultra-puissante (400 MHz) et possède des capacités vidéo, mais elle n'a **aucune connectivité sans fil** intégrée. Elle est destinée aux applications industrielles ou aux interfaces locales.

Comment reconnaître l'interface WiFi sur votre carte ?

Regardez le module métallique (souvent argenté) sur votre carte : si vous voyez une ligne en forme de "S" ou de "zig-zag" tracée sur le circuit imprimé ou une petite pièce en céramique à une extrémité, c'est l'antenne WiFi.



Le protocole **ESP-NOW** permet de communiquer entre cartes ESP32 par WiFi sans nécessiter la connection à un routeur.

Étapes d'une connexion complète

Lancer **WiFi.begin** ne suffit pas à garantir que vous êtes connecté. Le processus est asynchrone. Voici comment structurer une connexion propre :

Initialisation du mode Station

Avant de se connecter, il est crucial de dire à l'ESP32 de se comporter comme un client (Station) et non comme un point d'accès.

```
WiFi.disconnect \ Optionnel : nettoie les anciennes sessions  
1 WiFi.mode    \ 1 = Mode Station (STA)
```

Une fois **WiFi.begin** lancé, vous devez surveiller le statut.

```
z" SSID_RESEAU" z" PASSWORD" WiFi.begin  
  
: wait-wifi ( -- )  
  begin  
    WiFi.status 3 <> \ 3 correspond à WL_CONNECTED  
  while  
    ." En attente du WiFi..." cr  
    1000 ms  
  repeat  
    ." Connecté ! IP : " WiFi.localIP .ip cr ;
```

Le vocabulaire WiFi

Le vocabulaire **wifi** est accessible en exécutant **wifi vlist** :

```
WIFI_MODE_APSTA WIFI_MODE_AP WIFI_MODE_STA WIFI_MODE_NULL WiFi.config
WiFi.begin WiFi.disconnect WiFi.status WiFi.macAddress WiFi.localIP
WiFi.mode WiFi.setTxPower WiFi.getTxPower WiFi.softAP WiFi.softAPIP
WiFi.softAPBroadcastIP WiFi.softAPNetworkID WiFi.softAPConfig
WiFi.softAPdisconnect WiFi.softAPgetStationNum WiFi.enableIPv6
WiFi.linkLocalIPv6 WiFi.globalIPv6
```

WIFI_MODE_AP (-- 2)

Constante. Valeur 2.

Dans l'univers de l'ESP32 et de Forth, **WIFI_MODE_AP** (Access Point) est l'opposé du mode Station. Au lieu de se connecter à une box internet, l'ESP32 devient lui-même la "box".

Il crée son propre réseau WiFi auquel vous pouvez connecter votre smartphone ou votre ordinateur :

- **Configuration initiale** : Permet de configurer la carte ESP32 sans connaître les identifiants WiFi du lieu.
- **Contrôle direct** : Idéal pour piloter un robot ou un objet connecté en plein milieu d'un champ, sans routeur à proximité.
- **Interface locale** : Héberger une page Web de contrôle accessible via une IP fixe.

Utilisation :

```
WIFI_MODE_AP WiFi.mode      \ Définit le mode sur WIFI_MODE_AP
z" Mon_ESP32_Reseau"        \ SSID (Nom du réseau)
z" motdepasse123"           \ Mot de passe (min 8 caractères)
WiFi.softAP                  \ Notez l'utilisation de softAP au lieu de begin
```

Pour un point d'accès, on utilise souvent **WiFi.softAP** plutôt que **WiFi.begin**. La commande **WiFi.begin** cherche à rejoindre un réseau, tandis que **WiFi.softAP** le crée.

Lorsque l'ESP32 est en mode AP, il possède des caractéristiques par défaut :

- **Adresse IP par défaut** : C'est presque toujours **192.168.4.1**.
- **Nombre de clients** : Généralement limité à 4 ou 8 appareils simultanés sur un ESP32.
- **Canal (Channel)** : Par défaut sur le canal 1 (peut être modifié pour éviter les interférences).

WIFI_MODE_APSTA (-- 3)

Constante. Valeur 3.

Le mode `WIFI_MODE_APSTA` (souvent associé à la valeur **3**) est le mode le plus polyvalent de l'ESP32. Il permet à la puce de fonctionner simultanément comme une **Station (STA)** et comme un **Point d'Accès (AP)**.

C'est comme si votre ESP32 avait deux cartes réseau virtuelles partageant la même antenne physique.

Ce mode est la base de deux fonctionnalités célèbres dans le monde de l'IoT :

- **Le Portail Captif de Configuration** : Votre ESP32 essaie de se connecter à votre box (STA). S'il échoue, il active son propre réseau (AP) pour que vous puissiez vous y connecter avec votre téléphone et lui donner les nouveaux identifiants via une page web, sans jamais perdre le contact.
- **Le Répéteur WiFi (Range Extender)** : L'ESP32 reçoit le signal internet d'un côté et le rediffuse de l'autre pour étendre la portée du réseau.
- **Passerelle (Gateway)** : Il peut collecter des données de capteurs connectés à son propre réseau AP et les envoyer vers un serveur distant via sa connexion STA.

Ce mode est la base de deux fonctionnalités célèbres dans le monde de l'IoT :

- **Le Portail Captif de Configuration** : Votre ESP32 essaie de se connecter à votre box (STA). S'il échoue, il active son propre réseau (AP) pour que vous puissiez vous y connecter avec votre téléphone et lui donner les nouveaux identifiants via une page web, sans jamais perdre le contact.
- **Le Répéteur WiFi (Range Extender)** : L'ESP32 reçoit le signal internet d'un côté et le rediffuse de l'autre pour étendre la portée du réseau.
- **Passerelle (Gateway)** : Il peut collecter des données de capteurs connectés à son propre réseau AP et les envoyer vers un serveur distant via sa connexion STA.

Pour l'activer, on empile la valeur **3** avant d'appeler `WiFi.mode` :

```
WIFI_MODE_APSTA WiFi.mode          \ Active le mode hybride AP+STA

\ Configuration de la partie AP (son propre réseau)
z" Mon_Reseau_Local" z" mdp12345" WiFi.softAP

\ Configuration de la partie STA (se connecte à la box)
z" Ma_Box_Internet" z" mdp_secret" WiFi.begin
```

Bien que très puissant, ce mode a quelques limites à connaître :

- **Le Canal (Channel)** : Comme il n'y a qu'une seule antenne, le mode AP et le mode STA doivent impérativement utiliser le **même canal WiFi**. Si votre box

change de canal, l'ESP32 devra ajuster son propre réseau AP pour suivre, ce qui peut déconnecter brièvement les appareils qui y sont reliés.

- **Consommation** : C'est le mode qui consomme le plus d'énergie puisque la radio est sollicitée pour maintenir deux types de connexions.
- **Adressage** : L'ESP32 aura deux adresses IP différentes : une sur le réseau de la box (via **WiFi.localIP**) et une pour son propre réseau (via **WiFi.softAPIP**).

Dans ce mode, vous pouvez interroger les deux identités :

```
WiFi.localIP .ip    \ L'adresse donnée par votre box (ex: 192.168.1.45)
WiFi.softAPIP .ip   \ L'adresse de sa propre bulle WiFi (ex: 192.168.4.1)
```

C'est le mode "couteau suisse" par excellence pour rendre vos projets robustes et faciles à configurer !

WIFI_MODE_NULL (-- 0)

Constante. Valeur 0.

Le mode **WIFI_MODE_NULL** est, comme son nom l'indique, l'état "zéro" ou "vide" de l'interface WiFi de l'ESP32.

C'est le mode par défaut au démarrage (avant toute configuration) ou un mode de désactivation. Lorsqu'il est actif :

- L'interface WiFi n'est ni en mode Station (STA), ni en mode Point d'Accès (AP).
- La pile TCP/IP est virtuellement arrêtée pour la partie sans fil.
- L'ESP32 ne cherche pas à se connecter et n'émet aucun signal.

Il y a deux raisons principales pour appeler ce mode :

- **Économie d'énergie** : Le WiFi est le composant le plus gourmand de l'ESP32. Passer en **WIFI_MODE_NULL** (souvent via la commande **0 WiFi.mode**) permet de couper la radio et de réduire drastiquement la consommation électrique si votre projet doit effectuer des calculs locaux sans communiquer.
- **Réinitialisation propre** : Avant de changer radicalement de configuration WiFi (passer d'un mode AP complexe à un mode STA), certains développeurs forcent le mode NULL pour s'assurer que les ressources mémoires du WiFi sont libérées.

```
WIFI_MODE_NULL WiFi.mode \ Désactive l'interface WiFi
```

Ne confondez pas **WIFI_MODE_NULL** avec le mode **Power Save**. Le mode NULL coupe l'interface, tandis que le Power Save maintient la connexion avec le routeur tout en s'endormant entre deux balises (beacons).

C'est le mode "silence radio" total. Utile si votre ESP32 fonctionne sur batterie et n'a besoin d'envoyer des données qu'une fois par heure, par exemple.

WIFI_MODE_STA

Constante. Valeur 1.

WiFi.begin (ssid-z password-z --)

la commande **WiFi.begin** est le point d'entrée pour connecter la carte ESP32 à un réseau local. FORTH traite cette commande de manière interactive, ce qui permet de tester sa connectivité en temps réel via le terminal.

Le mot **WiFi.begin** attend deux chaînes de caractères (Z-strings) sur la pile : le **nom du réseau (SSID)** et le **mot de passe** :

```
z" Mon_SSID" z" Mon_Mot_De_Passe" WiFi.begin
```

Le SSID est le nom du routeur tel qu'il est diffusé sur votre réseau WiFi local. Le mot de passe est celui associé à ce SSID.

WiFi.config (ip dns gateway subnet --)

Dans l'environnement ESP32forth, **WiFi.config** est la commande qui vous permet de reprendre le contrôle sur l'adressage réseau. Par défaut, l'ESP32 utilise le **DHCP** (il demande une adresse IP automatiquement à votre box), mais **WiFi.config** permet de lui assigner une **IP statique** (fixe).

C'est essentiel si vous créez un serveur web sur votre ESP32, car cela évite que l'adresse IP ne change à chaque redémarrage.

La commande **WiFi.config** attend généralement plusieurs adresses IP (sous forme d'entiers 32 bits) sur la pile.

WiFi.disconnect (--)

La commande **WiFi.disconnect** est l'interrupteur logiciel qui permet de rompre proprement la liaison sans fil de l'ESP32. Bien qu'elle paraisse simple, elle joue plusieurs rôles cruciaux dans la gestion de la stabilité et de la consommation de votre module.

Lorsqu'elle est appelée, l'ESP32 envoie un paquet de "désassociation" au routeur (ou au client s'il est en mode AP).

- Elle stoppe les tentatives de reconnexion automatique.
- Elle libère une partie des ressources mémoire allouées à la session active.
- Elle fait basculer le **WiFi.status** vers la valeur **6 (WL_DISCONNECTED)**.

Il y a trois scénarios principaux :

- **Changement de réseau** : Avant de lancer un nouveau **WiFi.begin** sur un autre SSID, il est fortement recommandé de faire un **WiFi.disconnect** pour éviter que l'ancienne session ne crée des conflits.
- **Économie d'énergie** : Si votre projet n'envoie des données qu'une fois par heure, faire un **WiFi.disconnect** permet de couper la radio et d'économiser la batterie entre deux envois.
- **Réinitialisation (Reset)** : Si la connexion est instable ou "gelée", un appel à **WiFi.disconnect** suivi d'un délai (**1000 ms**) et d'un nouveau **WiFi.begin** est la méthode standard pour forcer une reconnexion propre.

En Forth, la commande est simple et ne prend pas de paramètres sur la pile :

```
WiFi.disconnect
```

Avant de commencer une nouvelle configuration, voici une bonne pratique :

```
: wifi-reset ( -- )
  WiFi.disconnect
  500 ms
  0 WiFi.mode      \ Passe en mode NULL pour tout effacer
  500 ms
  ." WiFi réinitialisé." cr
;
```

WiFi.enableIPv6

WiFi.localIP (-- ip)

La commande **WiFi.localIP** est un mot de type "getter" : elle ne prend rien sur la pile, mais elle y dépose l'adresse réseau actuellement attribuée à votre ESP32 par le routeur.

Lorsque vous appelez **WiFi.localIP**, ESP32forth interroge sa pile TCP/IP interne (LwIP).

- **Entrée** : Rien (--).
- **Sortie** : Un entier de 32 bits représentant l'IP (**ip**).

L'adresse est stockée sous forme de **4 octets packés** dans un seul nombre. Par exemple, si votre IP est **192.168.1.15**, le nombre sur la pile ne sera pas "192", mais une valeur brute comme **251766976** (en décimal) ou **\$0F01A8C0** (en hexadécimal).

WiFi.macAddress (addr --)

Le mot **WiFi.macAddress** permet de récupérer l'identifiant physique unique de votre puce ESP32. Contrairement à l'adresse IP (qui peut changer selon le réseau), l'adresse MAC est "gravée" en usine dans le matériel.

MAC signifie *Media Access Control*. C'est une suite de **6 octets** (souvent affichés en hexadécimal) qui sert de numéro de série mondial unique pour chaque interface réseau.

- **Format type : 24:0A:C4:XX:XX:XX**
- Les trois premiers octets identifient le constructeur (Espressif Systems dans notre cas).

Exemple d'utilisation :

```
\ store current mac Address
create myMac 6 allot

\ get mac address for current ESP32 card and store result in myMac
: getMyMac ( -- )
  myMac WiFi.macAddress
;

\ display MAC address in hex format
: .mac { mac-addr -- }
  base @ hex
  6 0 do
    mac-addr i + c@ <# # # #> type
    i 5 < if
      [char] : emit
    then
  loop
  base !
;
```

Utilité

1. **Identification unique** : Si vous avez 10 capteurs ESP32 sur un réseau, l'adresse MAC est le meilleur moyen de savoir "qui est qui" côté serveur.
2. **Sécurité (Filtrage MAC)** : Si votre routeur est configuré pour n'accepter que certains appareils, vous devez lui donner cette adresse.
3. **Génération d'ID** : On l'utilise souvent pour créer un nom de réseau WiFi (SSID) unique pour chaque appareil, par exemple **MonRobot-240AC4**.

La carte ESP32 possède en réalité **deux** adresses MAC distinctes (souvent décalées d'une unité) :

- Celle de l'interface **Station** (utilisée avec **WiFi.macAddress**).

- Celle de l'interface **SoftAP** (utilisée avec **WiFi.softAPmacAddress**).

WiFi.mode (n --)

Le mot **WiFi.mode** est le sélecteur de configuration matérielle de votre ESP32. Avant de tenter de vous connecter ou de créer un réseau, vous devez impérativement indiquer à la puce quel "rôle" elle doit jouer.

C'est une étape cruciale car elle initialise les piles logicielles et active la radio de manière appropriée.

Le mot attend un entier sur la pile qui correspond au mode souhaité,

Voici les quatre modes principaux que vous rencontrerez :

Valeur	Constante C++	Description
0	WIFI_MODE_NULL	Mode Off. Désactive le WiFi pour économiser l'énergie.
1	WIFI_MODE_STA	Station. L'ESP32 se connecte à un routeur (comme un smartphone).
2	WIFI_MODE_AP	Access Point. L'ESP32 crée son propre WiFi pour que d'autres s'y connectent.
3	WIFI_MODE_APSTA	Hybride. L'ESP32 fait les deux en même temps.

Si vous essayez de faire un **WiFi.begin** sans avoir fait **1 WiFi.mode**, la connexion risque d'échouer ou d'être instable car les ressources "Station" n'auront pas été allouées par le système d'exploitation (FreeRTOS) sous-jacent. Exemple :

```
: init-sta ( -- )
WIFI_MODE_STA WiFi.mode \ On passe en mode Station
z" Mon_SSID"
z" Mon_MDP"
WiFi.begin \ On lance la connexion
." Mode Station activé" cr ;
```

Le mode **WIFI_MODE_APSTA** est très puissant. Il permet par exemple à l'ESP32 de rester connecté à votre box internet tout en proposant un petit portail de secours si vous perdez la connexion, ou d'agir comme un répéteur WiFi.

Si vous changez de mode en cours d'exécution, il est souvent préférable de faire un petit **WiFi.disconnect** ou d'attendre quelques millisecondes pour laisser le temps à la puce de réinitialiser la transmission radio.

WiFi.getTxPower (-- powerx4)

Le mot `WiFi.getTxPower` permet de mesurer la puissance en émission radio de votre ESP32. Il retourne la puissance d'émission actuelle de l'antenne.

La puissance de transmission (TX Power) détermine la portée de votre signal WiFi.

- Elle s'exprime en **dBm** (décibel-milliwatts).
- Sur un ESP32, cette valeur se situe généralement entre **8 dBm** (puissance faible) et **20 dBm** (puissance maximale).

Le mot ne prend rien sur la pile et retourne un entier représentant la puissance.

```
WiFi.getTxPower .
```

1. **Diagnostic de portée** : Si vous n'arrivez pas à capter votre ESP32 à plus de 2 mètres, vérifiez avec `getTxPower` si la radio n'est pas bridée par logiciel.
2. **Gestion de l'énergie** : Plus la puissance est élevée, plus l'ESP32 consomme de courant (ampères) et plus il chauffe.
3. **Conformité** : Dans certains pays ou environnements sensibles, il peut être nécessaire de limiter la puissance pour respecter les normes locales ou ne pas brouiller d'autres appareils.

Il existe souvent le mot jumeau `WiFi.setTxPower` qui permet de modifier cette valeur. Par exemple, pour réduire la consommation :

```
: low-power-wifi ( -- )  
40 WiFi.setTxPower \ Définit une puissance plus faible  
." Puissance réglée à : " WiFi.getTxPower . cr ;
```

La puissance maximale réelle dépend aussi de la qualité de l'alimentation électrique de votre carte. Si votre alimentation est faible, monter la puissance TX peut provoquer un crash de l'ESP32 lors de l'émission des données.

WiFi.setTxPower (powerx4 --)

WiFi.softAP

WiFi.softAPBroadcastIP

WiFi.softAPConfig

WiFi.softAPdisconnect

WiFi.softAPIP

WiFi.softAPNetworkID

WiFi.softAPgetStationNum

WiFi.status

WiFi.linkLocalIPv6

WiFi.globalIPv6

Index lexical

WIFI_MODE_AP.....	6	WiFi.getTxPower.....	13
WIFI_MODE_APSTA.....	7	WiFi.localIP.....	10
WIFI_MODE_NULL.....	8	WiFi.macAddress.....	10
WIFI_MODE_STA.....	9	WiFi.mode.....	12
WiFi.begin.....	9	WiFi.setTxPower.....	13
WiFi.config.....	9	WiFi.softAP.....	13
WiFi.disconnect.....	9	WiFi.softAPBroadcastIP.....	13
WiFi.enableIPv6.....	10		9

